Tree-based Models for Nowcasting

Decision trees, Random Forests, and Gradient Boosted Trees — with an application to Jamaica's Real GDP Growth

Jose Luis Saboin, PhD
Inter-American Development Bank

Real GDP Nowcasting Tool for BoJ

May 5, 2026

"How a sequence of yes/no questions becomes a forecasting machine."

Part I — Decision Trees

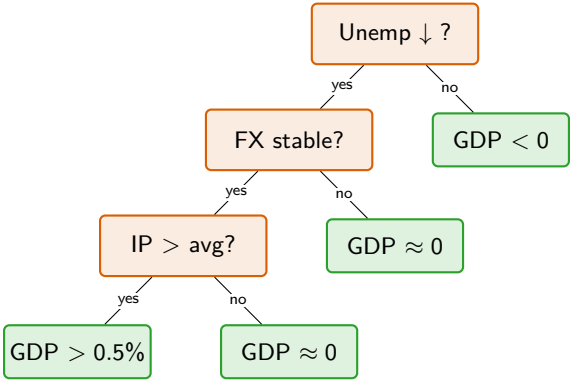
What is a decision tree?

A decision tree predicts an outcome by asking a **sequence of yes/no questions** about the input.

Toy nowcasting example. Predict whether next-quarter GDP will be above 0.5%:

- Q1: *Has unemployment fallen this quarter?*
- Q2 (if yes): *Is the JMD/USD rate stable?*
- Q3 (if yes): *Is industrial production above its 12-month avg?*

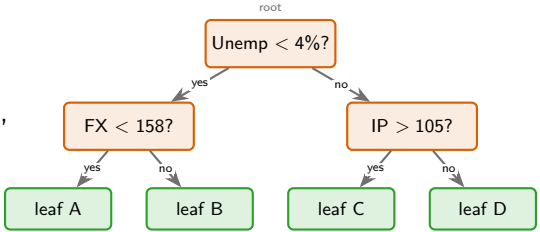
Each path through the tree ends at a **leaf** that gives the prediction.



Anatomy of a tree

- **Root node** — the first question; sees *all* training examples.
- **Internal nodes** — each is a question of the form "feature $j >$ threshold τ ?"
- **Branches** — the two answers (*yes* / *no*).
- **Children** — the two nodes below a split: left (L) = "yes", right (R) = "no".
- **Leaves** — terminal nodes (no children) that emit a prediction.

Each leaf corresponds to a *rectangular region* in feature space; the prediction in that region is constant.



4 leaves ⇒ 4 rectangular regions ⇒ 4 distinct GDP predictions.

Regression trees — the leaf is a number

Nowcasting Jamaica RGDP growth uses **regression** trees: each leaf emits the **mean** of the training GDPs that fall in it.

Concretely, for any new quarter:

1. Drop its feature row down the tree, following the yes/no answers at each split.
2. Land in some leaf.
3. The forecast is the average GDP of the training quarters that landed in that leaf during fitting.

At every split, the build process minimises the **weighted child MSE** (or MAE) so leaves end up holding training quarters with similar GDPs — making the leaf-mean a sharp prediction. The rest of this deck assumes regression throughout.

How does the tree decide *where* to split?

At each node, the tree tries *every* candidate (j, τ) — feature j, threshold τ — and picks the pair that minimises an **impurity** criterion in the two children.

For regression (nowcasting Jamaica RGDP growth): minimise weighted child MSE.

$$\mathcal{L}(j, \tau) \;=\; \frac{|S_L|}{|S|}\,\mathrm{MSE}(S_L) \;+\; \frac{|S_R|}{|S|}\,\mathrm{MSE}(S_R)$$

where:
S — training examples at the current node;
S_L — examples sent to the *left* child (the **"yes"** branch, i.e. those satisfying "feature $j \leq \tau$");
S_R — examples sent to the *right* child (the **"no"** branch);
$\mathrm{MSE}(S) = \frac{1}{|S|}\sum_{i \in S}(y_i - \bar{y}_S)^2$ — variance around the child's mean prediction.

Greedy: the tree picks the locally best split at each node and never reconsiders — fast, but not globally optimal.

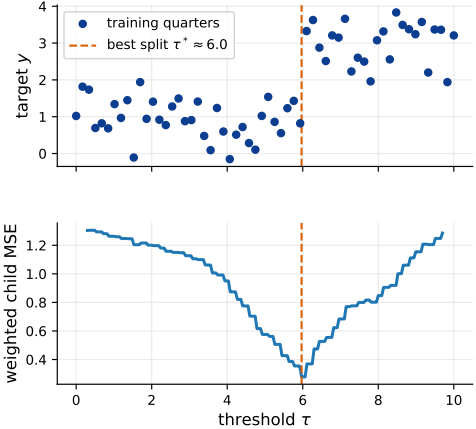
Building the tree top-down

Recursive algorithm.

1. Start at the root with *all* training examples.
2. Find the best split (j, τ) by minimising impurity.
3. Partition examples into S_L and S_R.
4. Recurse on each child.
5. Stop when:
   - the node is *pure* (all y's the same), or
   - `max_depth` is reached, or
   - $|S| < \texttt{min_samples_split}$.

What "best split" looks like for one feature.

Try every threshold; pick the one with lowest child MSE



The optimal threshold τ^* is wherever the weighted child MSE bottoms out as τ slides across the feature range.

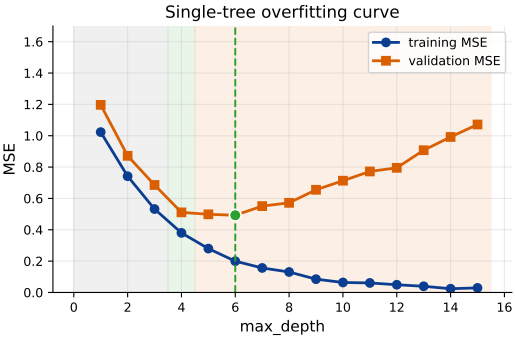
Trees overfit easily — and how to stop them

A deep enough tree will perfectly memorise the training set — one leaf per training example, training MSE $= 0$ — but **validation MSE explodes**.

Three knobs to tame this:

- `max_depth` — hard cap on tree depth.
- `min_samples_leaf` — refuse leaves with too few examples.
- `ccp_alpha` (cost-complexity) — penalise the tree for having many leaves.

For nowcasting Jamaica RGDP growth (~ 30 training quarters) one would set `max_depth` = 3–5 and `min_samples_leaf` ≥ 3.



Training MSE keeps falling with depth; validation MSE has a U-shape — pick the depth at the dip.

Pruning: two ways to stop a tree from overfitting

"Pruning" is the umbrella term for any rule that **shrinks the tree** so it generalises better. Two flavours:

	Pre-pruning (a.k.a. early stopping)	**Post-pruning** (cost-complexity)
When	While the tree is being *built*, top-down.	After the full tree is built, *bottom-up*.
How	Refuse to split a node if a stopping rule fires (depth cap, leaf too small, . . .).	Grow the tree fully, then snip back subtrees that don't justify their leaves.
Knobs (sklearn)	`max_depth`, `min_samples_split`, `min_samples_leaf`.	`ccp_alpha` — penalty α on #leaves; bigger α = more pruning.
Cost-complexity formula	—	$R_\alpha(T) = R(T) + \alpha\,\lvert T\rvert$, $R(T)$ = training error, $\lvert T\rvert$ = number of leaves. The pruner finds the subtree that minimises R_α.

For nowcasting Jamaica RGDP growth, pre-pruning carries most of the weight ($\sim$ 30 examples with `max_depth = 3–5` keeps the tree from running away in the first place). `ccp_alpha` is a finer second pass — often left at 0 in practice.

A single tree: pros and cons for nowcasting

Strengths	Weaknesses
▶ Easy to read & explain — a literal flowchart. ▶ Handles non-linearities and interactions automatically. ▶ No need to scale features or worry about correlated columns. ▶ Variable-importance scores fall out for free.	▶ **High variance** — a different bootstrap of the ~ 30 training quarters gives a very different tree. ▶ Predictions are *piecewise constant* — coarse for smooth time-series targets like GDP. ▶ Greedy splits miss globally better trees. ▶ Cannot extrapolate beyond the training range.

$\Rightarrow$ Single trees are rarely shipped alone for nowcasting. The strengths come from **ensembles of trees** — next two sections.

Part II — Bagging & Random Forests

Why ensemble many trees?

Single trees have **low bias** (they fit the training set tightly) but **high variance** (they're unstable to small data perturbations).

Averaging the predictions of B *independent* trees collapses variance:

$$\mathrm{Var}\left(\frac{1}{B}\sum_{b=1}^{B}\hat{y}_b\right) = \frac{\sigma^2}{B} \quad \text{(if the trees were truly independent)}$$

In reality the trees share training data, so they're *correlated* ($\rho > 0$), and the variance reduction is:

$$\mathrm{Var}(\bar{y}) = \rho\sigma^2 + \frac{(1-\rho)\sigma^2}{B}$$

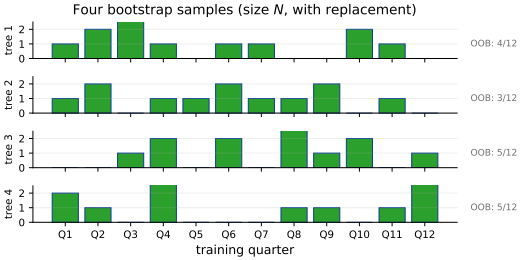
$\Rightarrow$ Two strategies to push ρ down: **bootstrap the training set** (**bagging**), and **randomise feature choice at each split** (**random forests**).

Bagging: bootstrap aggregation

Recipe.

1. Draw a *bootstrap sample* of size N *with replacement* from the N training quarters.
2. Fit a deep, unpruned tree on it.
3. Repeat B times $\Rightarrow$ B different trees.
4. For a new quarter: average the B predictions.

A bootstrap sample contains, in expectation, $\approx 63\%$ of the original training points; the other $\approx 37\%$ are **out-of-bag (OOB)** for that tree — free validation set.



Each bootstrap shuffles which quarters appear (and how many times) $\Rightarrow$ each tree sees a slightly different problem.

Random forests = bagging + random feature subsets

Bagging alone leaves the trees *too similar* — they all latch onto the same dominant indicator (e.g. unemployment). **Random forests** fix this by adding a second source of randomness:

At every internal split, the tree only considers a **random subset** of m features (out of the full p) when looking for the best split.

Default for regression: $m = p/3$.

Different splits along the tree see different feature subsets $\Rightarrow$ trees become structurally different $\Rightarrow$ correlation ρ drops $\Rightarrow$ more variance reduction from averaging.

For nowcasting Jamaica RGDP growth ($\sim$40 LARS-selected features), $m = 13$ would be the regression default.

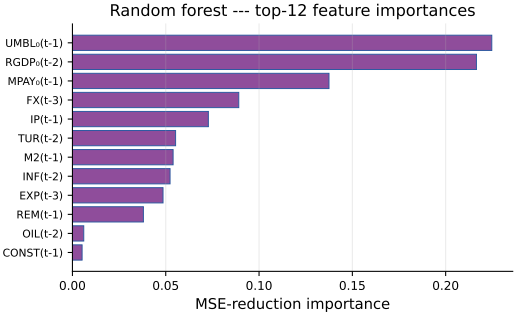
Random forest on Jamaica nowcasting

Setup.

- Every quarter-end row with non-NaN RGDP is one training example ($\sim$ 33 quarters).
- Each row flattened: K LARS-ranked features $\times$ `data_flat` lag-positions. Base specs use `data_flat` $= 3$ ($\Rightarrow K \times 3 \approx$ 120–150 columns); "flats" variants use `data_flat` $= 4$.
- `RandomForestRegressor`: `n_estimators` $= 130$, `max_features` $= 18$, `min_samples_leaf` $= 2$, `criterion = absolute_error`.
- Bagged across `ModelN` $= 200$ random seeds (variance-reduction layer on top of the bootstrap).

Free outputs from training:

- **OOB error** — predictive error on each quarter from the trees that didn't see it.
- **Variable importance** — average MAE drop attributable to each feature across all splits.



Top features ranked by importance — a sanity check on the LARS shortlist.

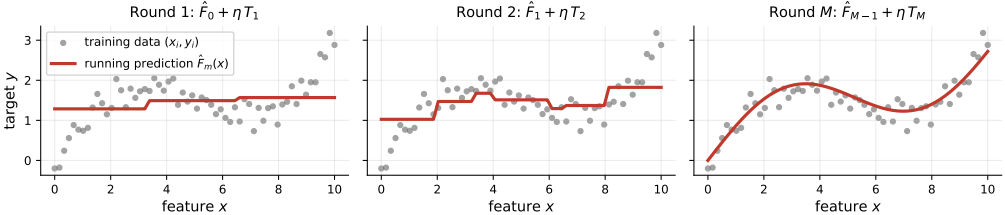
Part III — Boosting & Gradient Boosting

Boosting: fix the previous tree's mistakes, in sequence

Bagging trains trees **in parallel**, all on the original target. Boosting trains them **sequentially**, each new tree fitting the *residuals* (the part previous trees got wrong).

Recipe (regression, squared error): (1) start with $\hat{F}_0(x) = \bar{y}$; (2) compute residuals $r_i = y_i - \hat{F}_{m-1}(x_i)$; (3) fit a small tree T_m to $\{(x_i, r_i)\}$; (4) update $\hat{F}_m = \hat{F}_{m-1} + \eta\, T_m$; (5) repeat for M rounds.

where: $\hat{F}_m(x)$ — running prediction function after m rounds; $\bar{y}$ — training-target mean; r_i — residual of example i; $T_m(x)$ — the m-th tree, fitted to the residuals; η — learning rate; M — total boosting rounds.



Each round adds one more tree fitted to the leftover errors; the running prediction $\hat{F}_m$ tracks the data more tightly.

Gradient boosting: residuals are gradients

Squared-error residuals are exactly the negative gradient of MSE loss:

$$-\frac{\partial}{\partial \hat{F}(x_i)} \tfrac{1}{2}(y_i - \hat{F}(x_i))^2 \;=\; y_i - \hat{F}(x_i) \;=\; r_i$$

$\Rightarrow$ "fit residuals" generalises to "fit the negative gradient of any differentiable loss". That's **gradient boosting**.

General recipe.

1. Pick a loss $L(y, \hat{F})$ (squared error, log-loss, Huber, quantile, ...).
2. Initialise $\hat{F}_0$ to the constant minimising the loss on the training set.
3. For $m = 1, \ldots, M$: compute pseudo-residuals $g_i = -\partial L / \partial \hat{F}_{m-1}(x_i)$, fit a tree T_m to $\{(x_i, g_i)\}$, update $\hat{F}_m = \hat{F}_{m-1} + \eta\, T_m$.

η (learning rate) controls how big each correction step is: small η + many trees converges more reliably than large η + few trees.

What sklearn's `GradientBoostingRegressor` adds

Beyond the textbook recipe, sklearn's `GradientBoostingRegressor` adds:

Refinement	What it does
Subsampling per tree (`subsample`)	Each new tree is fitted on a random fraction of training rows — "stochastic gradient boosting", adds variance reduction.
Feature subsampling (`max_features`)	Random feature subset at each split, like Random Forests — decorrelates trees.
Cost-complexity pruning (`ccp_alpha`)	Each individual tree gets pruned to balance fit vs. size, beyond just `max_depth`.
Validation tracking (`n_iter_no_change`)	Built-in early stopping: monitors validation loss, halts when no improvement for k rounds.

Heavier industrial libraries (**XGBoost**, **LightGBM**) add histogram-based splits, regularised leaf weights (λ, γ), and leaf-wise growth — usually faster and slightly more accurate, but not used in the current pipeline.

GBT vs RF: when to use which

	Random Forest	**Gradient Boosted Trees**
Strategy	Reduce **variance** of independent deep trees.	Reduce **bias** of shallow trees *sequentially*.
Trees	Deep, unpruned, parallel.	Shallow (depth 3–6), sequential.
Sensitivity to tuning	Low — defaults usually fine.	High — learning rate, depth, and #trees interact.
Best-case accuracy	Very good.	Usually **the best on tabular data** when tuned.
Failure mode	Plateau at moderate accuracy.	Overfit if not early-stopped.
Pipeline pick	Fast benchmark, OOB diagnostics.	Production accuracy, with validation tracking.

For nowcasting Jamaica RGDP growth the natural pairing is: **RF** as a fast variance-reduced benchmark, **GBR** for the sharper sequential fit.

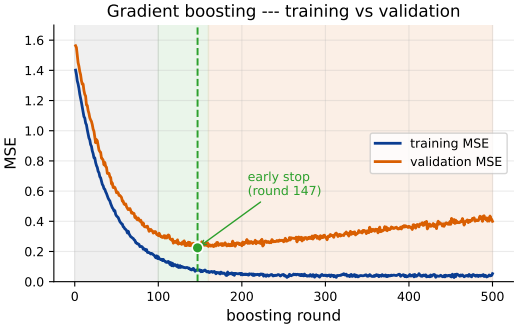
Gradient boosting on Jamaica nowcasting (sklearn)

Setup.

- Same flattened input as RF: every quarter-end row of K LARS-ranked features $\times$ `data_flat` lag-positions.
- `GradientBoostingRegressor`: `n_estimators` $= 100$, `learning_rate` $= \eta$, `loss = absolute_error`, `min_samples_split` $= 6$, `min_samples_leaf` $= 3$.
- Bagged across `ModelN` $= 200$ random seeds.

What you get.

- Loss curve has the canonical U-shape — x-axis is *boosting rounds* (each round adds one more tree).
- `feature_importances_` attribute identifies which lagged indicators carry signal.
- Bagging across the 200 seeds smooths single-run noise; `n_iter_no_change` (when set) provides built-in early stopping.



Validation loss bottoms out around the early-stopping round; past that, the model starts memorising training-quarter idiosyncrasies.

Part IV — Library knobs

Single-tree library knobs (sklearn `DecisionTreeRegressor`)

Parameter	Suggested	Intuition
`criterion`	`squared_error`	Loss to minimise at each split (regression).
`max_depth`	3–5	Max levels in the tree. Tiny data $\Rightarrow$ shallow tree.
`min_samples_split`	6	Don't try to split nodes with < 6 examples.
`min_samples_leaf`	3	A leaf must hold ≥ 3 examples.
`ccp_alpha`	0.01–0.05	Cost-complexity pruning strength.
`max_features`	`None`	Use all features for a single tree (only RF restricts).

Random Forest knobs (sklearn `RandomForestRegressor`)

Parameter	Suggested	Intuition
`n_estimators`	500	Number of trees. More = better, plateauing.
`max_features`	1/3	Random feature subset size at each split.
`bootstrap`	`True`	Sample training set with replacement per tree.
`oob_score`	`True`	Use out-of-bag examples as a free validation.
`max_depth`	`None`	Trees grow to purity (RF relies on averaging, not pruning).
`min_samples_leaf`	1–3	Slightly bigger leaves than single tree — still small.
`n_jobs`	−1	Use all CPU cores; trees are independent.

Gradient boosting knobs (sklearn `GradientBoostingRegressor`)

Parameter	Suggested	Intuition
`loss`	`squared_error`	Loss to minimise. `huber` for outlier-robust runs (Melissa quarter).
`n_estimators` (M)	500	Max boosting rounds; rely on early stop.
`learning_rate` (η)	0.05	Shrinks each tree's contribution. Small $\eta \times$ many trees $=$ smoother fit.
`max_depth`	3–5	Each tree is a *weak* learner. Keep shallow with ~ 30 training quarters.
`min_samples_leaf`	3	Leaves must hold ≥ 3 examples to be created.
`subsample`	0.8	Stochastic boosting: each tree fitted on a random 80% of rows.
`max_features`	1/3	Random feature subset at each split (analogue of RF).
`ccp_alpha`	0.0–0.05	Cost-complexity pruning of each individual tree.
`n_iter_no_change`	50	Halts boosting when validation loss stalls for 50 rounds.
`validation_fraction`	0.15–0.20	Fraction of training data held out internally for the early-stop check.

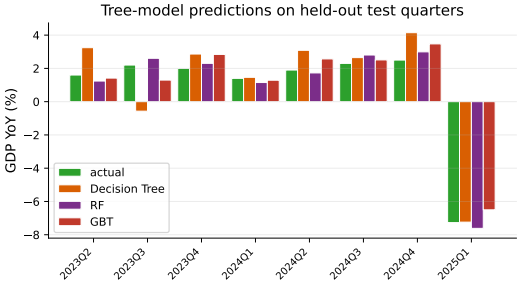
Part V — Tree models in the Jamaica pipeline

The training rows in tree-friendly form

Trees consume the data *flattened*: each quarter-end row of $\sim K$ LARS-ranked indicators is widened to also include the previous (`data_flat` $-$ 1) lagged copies of every column. The tree doesn't "see time"; lag structure is implicit in column names like `UMBL_2`.

Train / val / test split:

- Train: 2015-Q1 to 2023-Q1 ($\sim$ 33 quarters of non-NaN RGDP rows).
- Validation: held-out via `ModelN`-bagging across random seeds, plus OOB.
- Test: post 2023-Q2 (held out, never seen).



Decision-tree, RF, and GBT predictions on the held-out test quarters. Different smoothing strategies $\Rightarrow$ different errors.

Decision Tree vs RF vs GBT: comparison on Jamaica nowcasting

Aspect	**Decision Tree**	**Random Forest**	**Gradient Boosting**
Strategy	Single greedy split structure	Many independent trees, averaged	Many sequential trees, summed
Variance	*High* — unstable to small data shifts	*Low* — averaging decorrelates	Controlled by η + early stop
Tuning difficulty	Very low (4 knobs)	Low (defaults work)	Medium (η, depth, rounds)
Sample efficiency	Poor (overfits easily)	Good	Good with early stop
Best at ...	Interpretable baseline	Robust benchmark; OOB diagnostics	Sharpest tabular fit

Practical recommendation. The three sit on a spectrum from *one greedy tree* to *many independent trees averaged* to *many sequential trees summed*. In practice ensemble RF and GBT (e.g. simple average); the single decision tree is mainly a pedagogical baseline and a sanity-check.

Bias-variance recap across the tree models

Every supervised learner trades **bias** (systematic error from too-simple a hypothesis) against **variance** (sensitivity to which ~ 30 training quarters you trained on). The tree-family hits three different points on this curve:

	Bias	**Variance**
Single tree (deep)	Low	High — wildly different per bootstrap.
Random Forest	Same as single tree (low)	Much lower (averaging B trees $\Rightarrow \rho\sigma^2 + (1-\rho)\sigma^2/B$).
GBT (with early stop)	Lower over rounds (each tree shaves a bit)	Modest, controlled by η and early stop.

Both production-ready ensembles (**RF, GBT**) push variance down by some form of averaging or early stopping — because with ~ 30 training examples you cannot afford raw variance.

Summary

- A **decision tree** predicts via a recursive sequence of yes/no thresholds; greedy top-down build minimising weighted child MSE (or MAE) at each split.
- Single trees are interpretable but **high variance** on small data — never shipped alone for nowcasting.
- **Random Forest** = bagging + random feature subsets at each split. Variance reduction by averaging many decorrelated deep trees. OOB error is a free validation set.
- **Gradient Boosting** = sequential trees, each fitting the gradient (residuals for MSE) of the running prediction. With early stopping (`n_iter_no_change`), sklearn's `GradientBoostingRegressor` hits the lowest tabular-data error in practice.
- For **Jamaica nowcasting** ($\sim$30 quarterly examples, $\sim$40 LARS features), every tree model uses the same 12-month sliding-window setup, flattened to a $\sim$480-column input row.
- **Best practice**: deploy RF and GBT side-by-side — their disagreements are themselves informative.